Десислава - консултация УК

Линк:
https://drive.google.com/drive/folders/1dnDl1HueIaIh0eXATzSCSsCvodI39Ly5

На изпита **1) се описва самия процес на тестван**е (как се извършва тестването като техника с последователност от стъпки) и **2) прилагане на техниката върху конкретен пример и генериране на тестовите сценарии**:

Техники

Тестване по двойки

Идеята е да не се правят всички тестови сценарии на входа, а тестовите сценарии да се генерират на двойки стойности.

При техниката като помощно средство се ползва ортогонален масив:

Ортогонален масив

- Пример: ортогонален масив
  - Масив с размерност 4 x 3 и елементите със стойности 1 или 2
  - Налице са всички възможни комбинации на стойностите на двойки елементи
  - От 8 възможни комбинации на стойностите на всички елементи са налице само 4

$L_4(2^3)$

	Factors		
Runs	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

(1,1), (1,2), (2,1), (2,2)

(2,2,2)

- Обозначаване на ортогонален масив
  - $L_{Runs}(Levels^{Factors})$
    - Изпълнения (Runs): Брой на редовете в масива
    - Нива (Levels): Брой на възможните стойности на елементите в масива
    - Фактори (Factors): Брой на колоните в масива

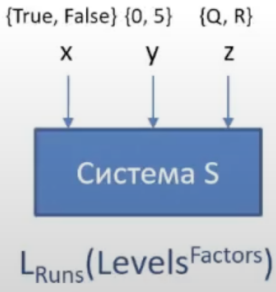
Всеки ортогонален масив се характеризира с **фактори** (брой колините в масива), **нива** (броят от възможните стойности на всеки елемент от масива) и **изпълнения** (брой редове в масива)

Когато имаме задача:

- Пример

$L_4(2^3)$

	Factors		
Runs	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1



$L_4(2^3)$

	Фактори		
Изпълнения	1	2	3
1	True	0	Q
2	True	5	R
3	False	0	R
4	False	5	Q

Пишем ЗАДЪЛЖИТЕЛНО **“Всеки ред от таблицата се генерира 1 тестов сценарий” -** за втората таблица, където са приложени стойностите.

начин за попълване на ортогонален масив:

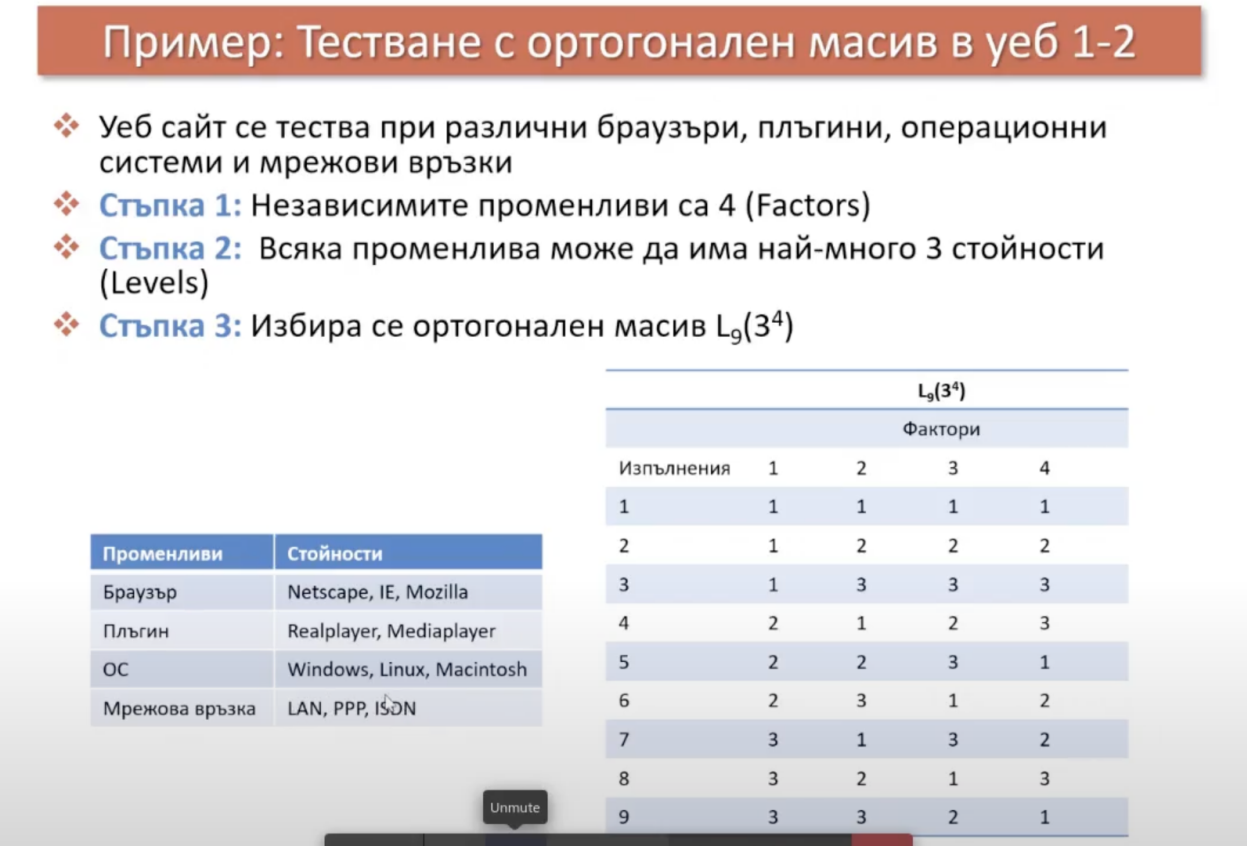
1ва колона: 111122223333

2ра колона: редуваме

почваме ред 1: само 1ци

всяка колона я почваме с 123, 123, останалите които остават ги разджуркваме

Стъпки:



Пример: Тестване с ортогонален масив в уеб 2-2

- **Стъпка 4:** Свързване на променливите с факторите и стойностите с нивата на масива
- **Стъпка 5:** Фактор 2 има три специфицирани нива в масива, но съответстващата му променлива (Плъгин) има само 2 възможни стойности (Realplayer и Mediaplayer). Неасоциираните нива се запълват със стойностите на променливата, като се редуват отгоре надолу
- **Стъпка 6:** Генерират се 9 тестови сценарии

Тестов сценарий	Браузър	Плъгин	ОС	Мрежова връзка
1	Netscape	Realplayer	Windows	LAN
2	Netscape	2 (Realplayer)	Linux	PPP
3	Netscape	Mediaplayer	Macintosh	ISDN
4	IE	Realplayer	Linux	ISDN
5	IE	2 (Mediaplayer)	Macintosh	LAN
6	IE	Mediaplayer	Windows	PPP
7	Mozilla	Realplayer	Macintosh	PPP
8	Mozilla	2 (Realplayer)	Windows	ISDN
9	Mozill	Mediaplayer	Linux	LAN

Unmute

Задача 07.2019 г.

Задача 3. Спецификация на софтуерна система за провеждане на обучения за получаване на допълнителна квалификация за учители изисква попълване на формуляр за заявка. Формулярът включва попълване на 4 полета за избор (ОКС, научно звание, специалност, тип идентификационен номер), като всяко от тях предлага следните опции:

- ОКС: Бакалавър, Магистър, Доктор
- Научно звание: Да, Не
- Специалност: Математика, Информатика, Компютърни науки
- Тип идентификационен номер (ИН): ЕГН, ЛНЧ, Друго

Да се дефинират тестови сценарии, покриващи избора на стойности от полетата за избор чрез прилагане на техниката за тестване по двойки с ортогонален масив. Да се опишат стъпките за прилагане на техниката и да се представят тестовите сценарии в таблица.

уу.07.2019 - УК

Стъпки за прилагане на техниката за тестване по двойки:

① Определяне на независими променливи: 4 (фактори)

② Всяка променлива има максимум 3 стойности levels: 3

③ Избираме ортогонален масив: $L_9(3)^4$

~~Тест сц~~ изпит ~~Изпълнения~~	1	2	3	4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

фактори

④ Свързване на променливите с факторите и стойностите с нивата на масива

⑤ Фактор 2 има 3 специфични нива, но изпитваната му пром. (~~Изпит~~ научно звание) има 2 възм. стойности. Несъществуващите нива се заместват със стойностите на променливата, които се редуват отгоре надолу.

Тестов сценарий	ОКС	научно звание	Специалност	Тип изпит. номер
1	Бакал.	Да	Мат.	ЕГН
2	Бакал.	Не	Инф	1НЧ
3	Бакал	(3) Да	КН	Др.
4	Маг	Да	инф	Др
5	Маг.	Не	КН	ЕГН
6	Маг.	(3) Не	Мат	1НЧ
7	~~Маг.~~ Д-р	Да	КН	1НЧ
8	Д-р	Не	Мат	Др
9	Д-р	(3) Да	Инф	ЕГН

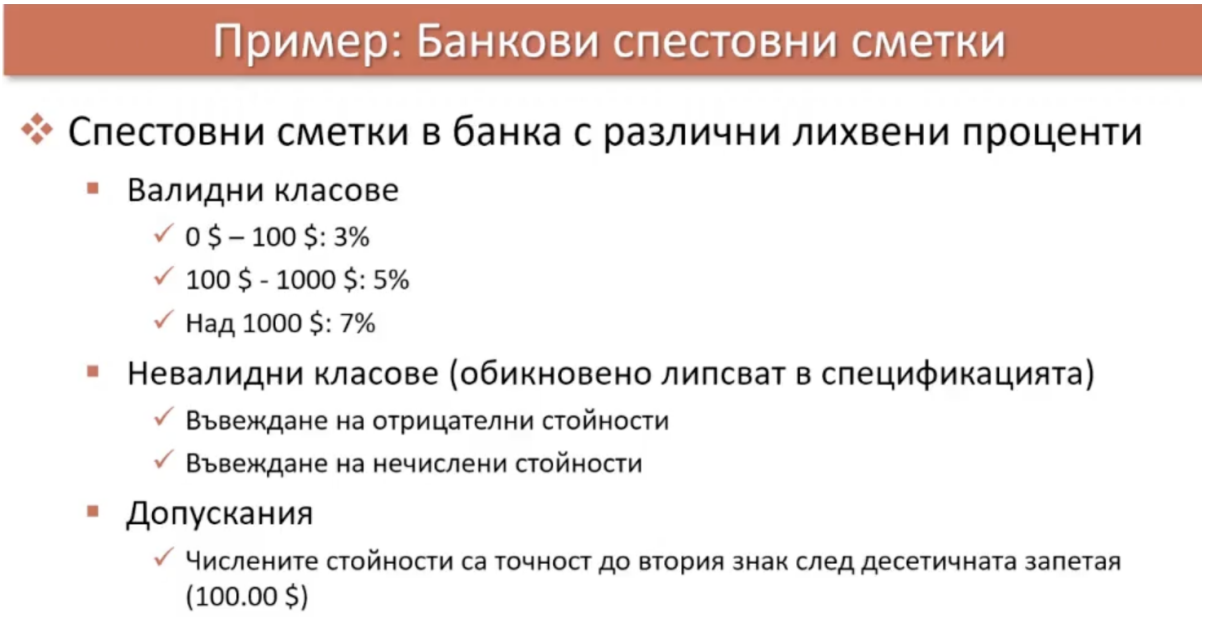
~~Задача~~

⑥ Генерирате a^1 тестови сц. да вит ред от таблицата.

▼ Класове на еквивалентност и гранични стойности

Разделяме входния домейн на класове и от всеки клас взимаме 1 стойност, с която тестваме. Разделянето става на база правилата, според които работи съответната система.

Пример:



Пример: Банкови спестовни сметки

Критерии са сравнение	Класове на еквивалентност	"Наивен" подход
Брой тествани класове	4	2
Брой тестове	4	16

Невалиден клас	Валиден клас (3%)	Валиден клас (5%)	Валиден клас (7%)	Входни тестови данни
-0.01$	0.00 $ 100.00 $	100.01 $ 999.99 $	1000.00 $	50.00 $
Входни тестови данни				100.00 $
-10.00 $	50.00 $	260.00 $	1348.00 $	150.00 $
Изходни данни				200.00 $
Съобщение за грешка	3%	5%	7%	...
				800.00 $

Трябва да се каже Има 4 тестови сценарии, с които тестваме 4те класа на еквивалентност и 6 тестови сценарии, с които тестваме границите

Задача 3. (10 точки) Софтуерна система е проектирана да приема резултати от независими тестове на машини. Всеки тест включва проверка на 5 операции. Всяка машина трябва да има изпълнени 5 индивидуални теста, като максималният резултат от всеки тест е 20. Системата изчислява и общия резултат от всички тестове. Резултатите от тестовете се сравняват по двойки. Машината се подлага на следващи тестове при следните условия:

- Ако два теста имат разлика по-голяма от 2 при проверка на една и съща операция;
- Ако разликата в крайните резултати от 2 теста е по-голяма от 10.

Опишете как ще се извърши тестването с класове на еквивалентност и анализ на граничните стойности, като включите следните стъпки:

1. Определете валидни и невалидни класове на еквивалентност.
2. Определете гранични стойности за тестване.
3. Дефиниране на тестови сценарии с конкретни стойности. Какъв е минималният брой на тестовите сценарии, които покриват класовете на еквивалентност и граничните стойности?

Зад. 3 08.09.2021г.

Разделяме входния домейн на класове на база правилата, според които работи системата.

Разглеждаме само първото условие, за което машината се подлага на следващи тестове, понеже ако на всяка от 5те операции има разлика точно 2 при сравняване на 2 теста, то разликата в крайните резултати е точно 10, при което машината не се подлага на следващи тестове. ∧ Това означава, че усл. 2 няма как да не е спазено, ако ~~всяка от~~ усл. 1 е спазено за ∀ операции.

⚠ Допускаме, че ∀ от 5 операции може да има ред. м/у 0 и 16.

Нека τ е разл. на една и съща операция при сравняване на 2 теста.

> За се напрви, колко τ е макс. разл. на една и съща операция при сравн. на ∀ 5 опер. б1τ

Валидни класове:

- $\tau \in [0, 2]$: машината не се подлага на сл. тестове
- $\tau \in [3, 16]$: машината се подлага на следващи тестове

Невалидни класове:

$\tau < 0$

$\tau > 16$

Допускаме: Числените стойности на τ са цели числа.

бел. Практична стойност 16 идва при ситуация, където имаме 1 опер. × 16τ и 4 опер. × 1τ. тогава макс. разлика може да е 16 (ако T_1 е 16 и T_2 е 0)

3.1

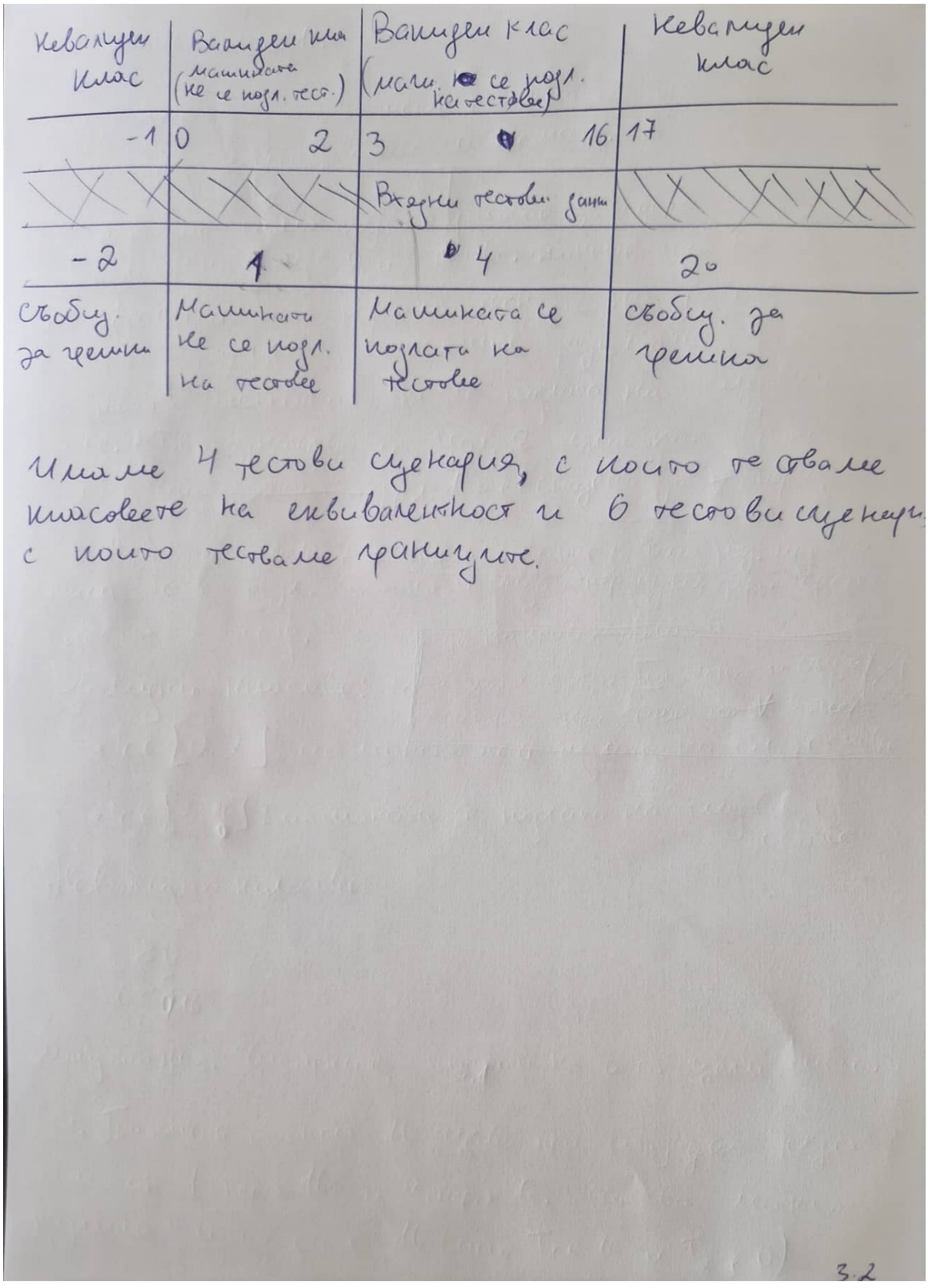

Невалиден клас
Валиден клас (машината не се подл. тест.)
Валиден клас (маш. се подл. на тестове)
Невалиден клас
-1
0
2
3
16
17
Входни тестови данни
-2
1
4
20
Свобод. за ремик
Машината не се подл. на тестове
Машината се подлага на тестове
Свобод. за ремика
Имаме 4 тестови сценария, с които тестваме класовете на еквивалентност и 6 тестови сценария, с които тестваме граничните.
3.2

Таблица на решенията

- Идентифициране на подходяща функционалност или подсистема с поведение, реагиращо на комбинация от входове или събития.
- Идентифицират се възможните условия.
- Идентифицират се комбинациите от True/False на условията.
- Идентифицира се правилният резултат (действие) за всяка комбинация.
- За всяка комбинация се създават тестови сценарии.

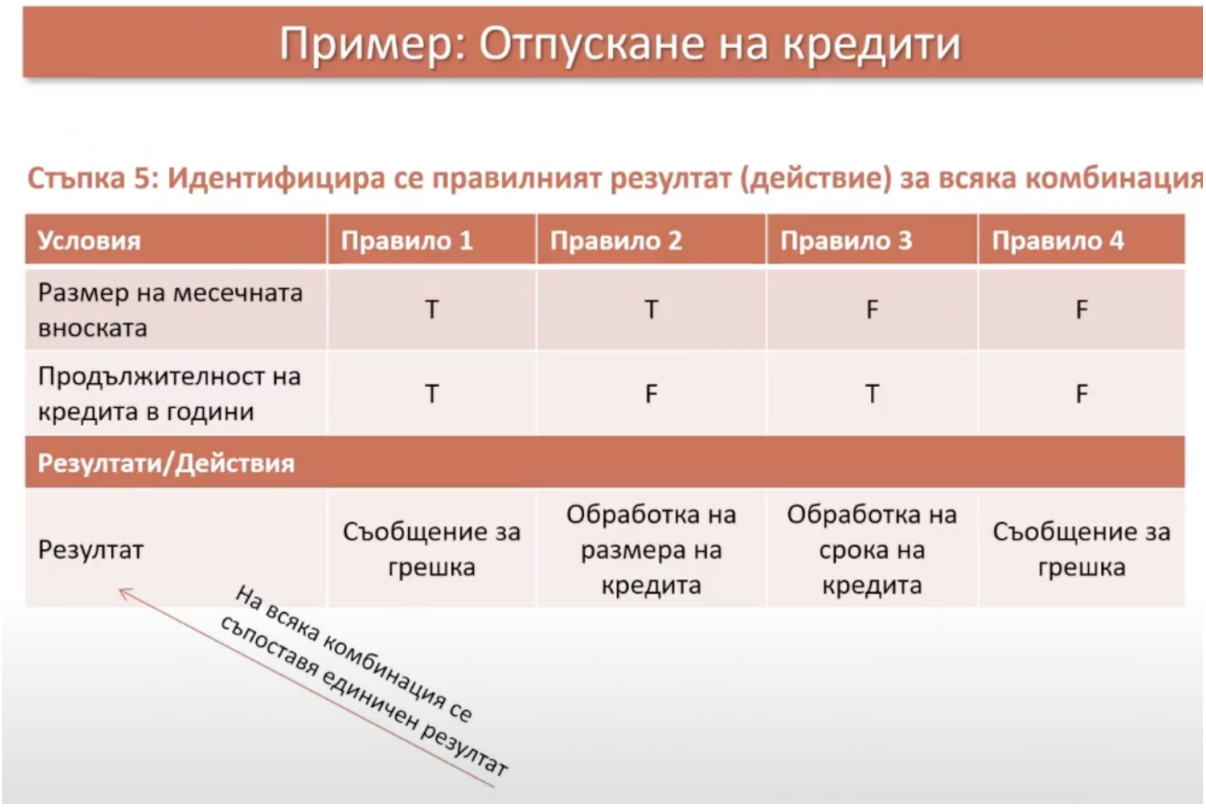
Дефиниране на таблица на решенията

- Условия: C_1, C_2, C_3
- Следствия: E_1, E_2, E_3
- Възможни стойности при решаване на предикатите от условията: Y, N, _
- Правила: $R_1 \div R_8$
  - Представят възможните комбинации на условията и следствията от изпълнението им
  - За всяко правило посредством индекс се задава и последователността на възникване на следствията
- Чек-сума
  - Използва се за верификация на комбинациите, които таблицата на решенията представя

		Правила							
Условия	**Стойности**	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8
C_1	Y, N, _	Y	Y	Y	Y	N	N	N	N
C_2	Y, N, _	Y	Y	N	N	Y	Y	N	N
C_3	Y, N, _	Y	N	Y	N	Y	N	Y	N
Следствия									
E_1		1		2	1				
E_2			2	1			2	1	
E_3		2	1	3		1	1		
Чек-сума	8	1	1	1	1	1	1	1	1

Правилата са 8, защото имаме 2^3, затова имаме 8 колони. Започваме да попълваме с Y/N, като първо се дели 8/2 = 4 → първите 4 колони са Y, след това първите 2 са Y и редуваме 2 по 2, винаги на последния ред редуваме Y/N.

Примери:



Условия	Правило 1	Правило 2	Правило 3	Правило 4
Размер на месечната вноската	T	T	F	F
Продължителност на кредита в години	T	F	T	F
Резултати/Действия				
Резултат	Съобщение за грешка	Обработка на размера на кредита	Обработка на срока на кредита	Съобщение за грешка

Пример: Отпускане на кредити

Стъпка 5: Идентифицира се правилният резултат (действие) за всяка комбинация.

Условия	Правило 1	Правило 2	Правило 3	Правило 4
Размер на месечната вноската	T	T	F	F
Продължителност на кредита в години	T	F	T	F
Резултати/Действия				
Резултат	Съобщение за грешка	Обработка на размера на кредита	Обработка на срока на кредита	Съобщение за грешка

На всяка комбинация се съпоставя единичен резултат

Пример: Кредитна карта

Условия	Правило 1	Правило 2	Правило 3	Правило 4	Правило 5	Правило 6	Правило 7	Правило 8
Нов клиент (15%)	T	T	T	T	F	F	F	F
Лоялен клиент (10%)	T	T	F	F	T	T	F	F
Ваучер за отстъпка (20%)	T	F	T	F	T	F	T	F
Действия								
Отстъпка (%)	X	X	20	15	30	10	20	0

Неосъществими комбинации

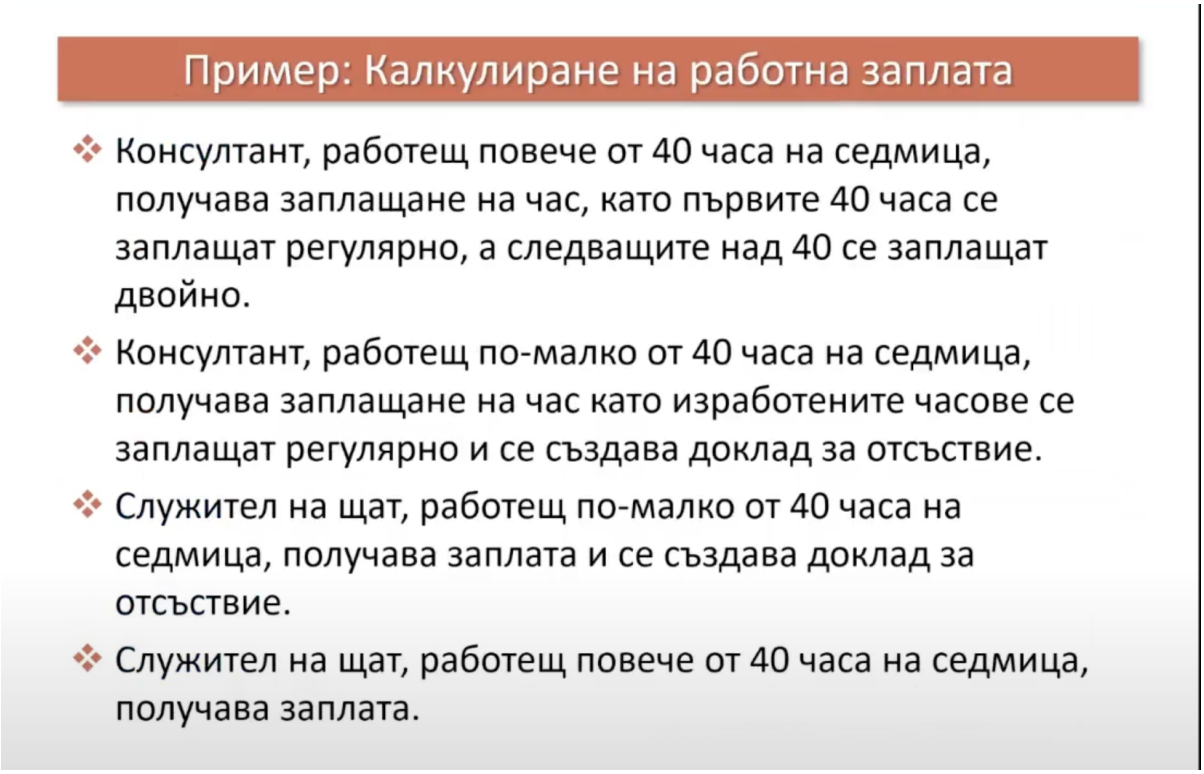
Ваучерът за отстъпка не е валиден заедно с отстъпката за нов клиент

- Всяка комбинация представлява отделен тест.
- При голям брой комбинации, комбинациите се приоритизират и се тестват само най-важните от тях.

Неосъществими комбинации: няма как клиентът да е нов и лоялен едновременно.

Неосъществими комбинации пишем САМО ако в условието е изрично казано, че системата трябва да отрази това по даден начин.

ИСТИНСКИ ПРИМЕР:



Пример: Тестване с таблица на решенията 1-3

- **Стъпка 1:** Идентифицират се следните условия и следствия
  - C_1: Служител на щат
  - C_2: Работещ по-малко от 40 часа
  - C_3: Работещ 40 часа
  - C_4: Работещ повече от 40 часа
  - E_1: Изплащане на заплата
  - E_2: Създаване на доклад за отсъствие
  - E_3: Регулярно заплащане на час
  - E_4: Двойно заплащане на час
- **Стъпка 2:** Условията и следствията се изброяват в таблица на решенията
- **Стъпка 3:** Броят на комбинациите е $2^4 = 16$
- **Стъпка 4:** Колоните на таблицата се попълват като се има предвид, че $RF_1 = 16/2 = 8$, $RF_2 = 8/2 = 4$, $RF_3 = 4/2 = 2$, $RF_1 = 2/2 = 1$

Your microphone is muted.

Пример: Тестване с таблица на решенията 2-3

		Правила															
Условия	**Стойности**	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
C_1	Y, N	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N
C_2	Y, N	Y	Y	Y	Y	N	N	N	N	Y	Y	Y	Y	N	N	N	N
C_3	Y, N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
C_4	Y, N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
Следствия																	
E_1																	
E_2																	
E_3																	
E_4																	

- **Стъпка 5:** Редуцират се следните правила
  - Ако C_1 = True и C_2 = True, то стойностите на условията C_3 и C_4 нямат значение. Обединяват се правила 1, 2, 3 и 4
  - Ако C_1 = True и C_2 = False, то стойностите на условията C_3 и C_4 нямат значение. Обединяват се правила 5, 6, 7 и 8
  - Ако C_1 = False и C_2 = True, то стойностите на условията C_3 и C_4 нямат значение. Обединяват се правила 9, 10, 11 и 12
  - Ако C_1 = False, C_2 = False и C_2 = True, то стойността на условието C_4 нямат значение. Обединяват се правила 13 и 14

- **Стъпка 6:** Изчисляване на чек-сумата за колоните на таблиците
- **Стъпка 7:** Определяне на следствията от всяко правило
  - Ако C_1 = True и C_2 = True, то са налице следствия E_1 и E_2, които се индексират съответно с 1 и 2 според реда на настъпването им.
- **Стъпка 8:** Генерират се 6 тестови сценарии, съответстващи на правилата в таблицата

		Правила					
Условия	**Стойности**	1	2	3	4	5	6
C_1	Y, N	Y	Y	N	N	N	N
C_2	Y, N	Y	N	Y	N	N	N
C_3	Y, N	-	-	-	Y	N	N
C_4	Y, N	-	-	-	-	Y	N
Следствия							
E_1		1	1				
E_2		2		2			
E_3				1	1	1	
E_4						2	
Чек-сума	16	4	4	4	2	1	1

Задача ДИ 2021 **таблица на решенията**:

Взаимно застрахователно дружество пуска своите акции на фондовата борса и предлага на своите членове премии при условие, че имат задължително „спестовна застраховка“ и са станали членове на дружеството преди 2015 г. Премиите могат да бъдат изплатени в брой или получени под формата на акции в ново търговско дружество. За членове, които имат „спестовна застраховка“, но са се присъединили към дружеството по-късно, е допустимо заплащане на премия в брой, но не и под формата на акции.

1. Създайте таблица за взимане на решения, която описва правилата, по които трябва да функционира софтуерна система, автоматизираща ситуацията по-горе. Опишете последователността от стъпки за създаване на таблицата.
2. Дефинирайте тестови сценарии, използвайки създадената таблица.
3. Какъв е очаквания резултат от следния тестов сценарий: Иван Петрова има „спестовна застраховка“ от 2017 г.?

Време за работа: 20-30 мин.

Стъпка 1: Идентифицират се следните условия и следствия

- C1: Текущ член на дружеството
- C2: Член на дружеството от 2015 г.
- C3: Притежател на „спестовна застраховка“
- E1: Плащане на премия в брой
- E2: Плащане на премия във вид на акции

Стъпка 2: Условията и следствията се изброяват в таблица на решенията

Стъпка 3: Броят на комбинациите е $2^3 = 8$

Стъпка 4: Колоните на таблицата се попълват като се има предвид, че RF1 = RF2 = 8/2 = 4, RF3 = 4/2 = 2, RF1

Стъпка 5: Определяне на следствията от всяко правило

Условия	**Стойности**	1	2	3	4	5	6	7	8
C_1	Y, N	Y	Y	Y	Y	N	N	N	N
C_2	Y, N	Y	Y	N	N	Y	Y	N	N
C_3	Y, N	Y	N	Y	N	Y	N	Y	N

Следствия								
E_1	Y	N	Y	N	N	N	N	N
E_2	Y	N	N	N	N	N	N	N

Стъпка 6: Редуцират се следните правила като се създава нова таблица

- Ако C_1 е False, то C_2 и C_3 нямат значение. Следователно могат да се обединят правила 5, 6, 7 и 8.
- Ако C_3 е False, то C_2 няма значение. Следователно могат да се обединят правила 2 и 4.

Условия	**Стойности**	1	2	3	4
C_1	Y, N	Y	Y	Y	N
C_2	Y, N	Y	-	N	-
C_3	Y, N	Y	N	Y	-
Следствия					
E_1		Y	N	Y	N
E_2		Y	N	N	N
Чек-сума	8	1	2	1	4

Стъпка 6: Изчисляване на чек-сумата за колоните на таблиците

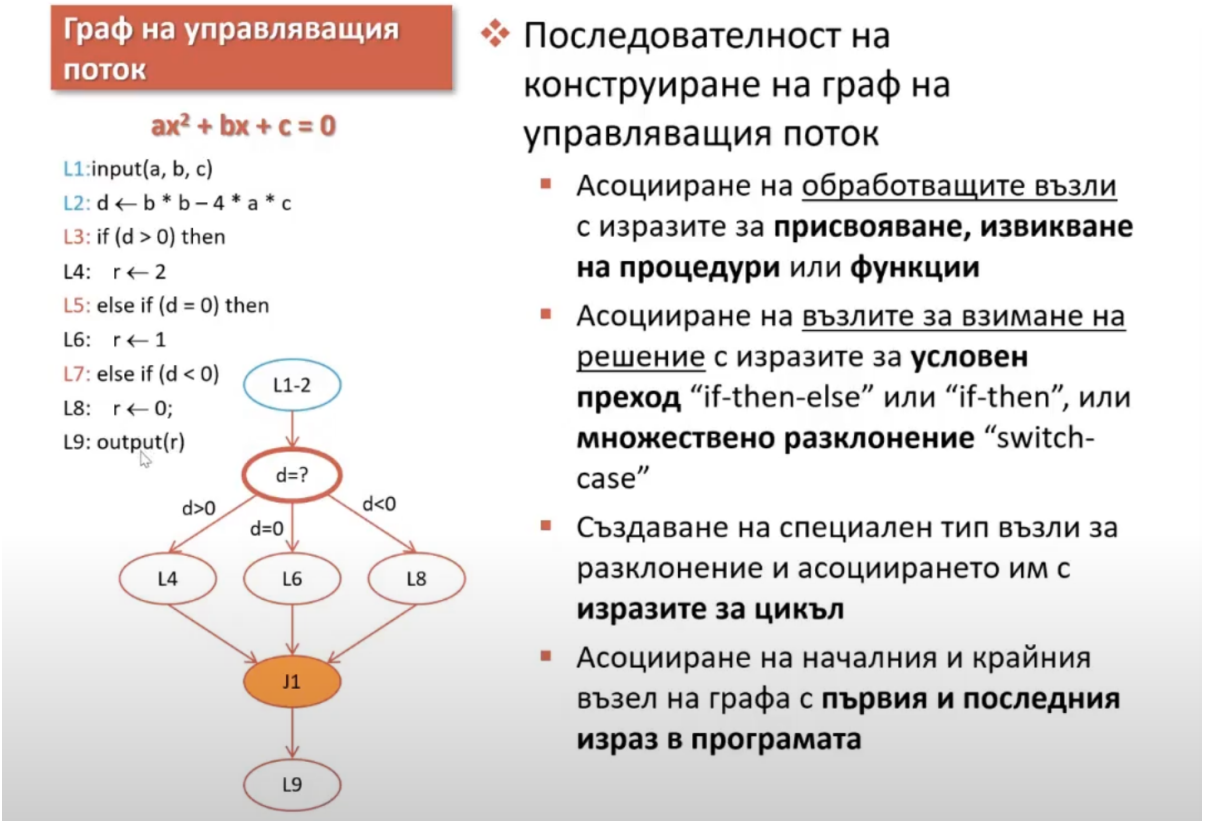
Your microphone is muted.

Стъпка 8: Генерират се 4 тестови сценарии, съответстващи на правилата в новата таблица

Иван Петров получава премия в брой.

Задача ДИ **таблица на решенията**:

Граф на управляващия поток



ВАЖНО: Ако има такава задача: да се направи граф и да се опишат тестовите сценарии т.е. в случая че има 3 тестови сценарии, които се определят от стойността на d и очакваните резултати. Свободен тест.

ВАЖНО ВАЖНО ВАЖНО

зад. ДИ 08.2020

Задача 3. Даден е следният псевдо-код:

```
Program BestInterest
Interest, Base Rate, Balance: Real

Begin
Base Rate = 0.035
Interest = Base Rate

Read (Balance)
If Balance > 1000
Then
 Interest = Interest + 0.005
 If Balance < 10000
 Then
  Interest = Interest + 0.005
 Else
  Interest = Interest + 0.010
 Endif
Endif

Balance = Balance * (1 + Interest)

End
```

а) Да се конструира граф на управляващия поток за псевдо-кода.

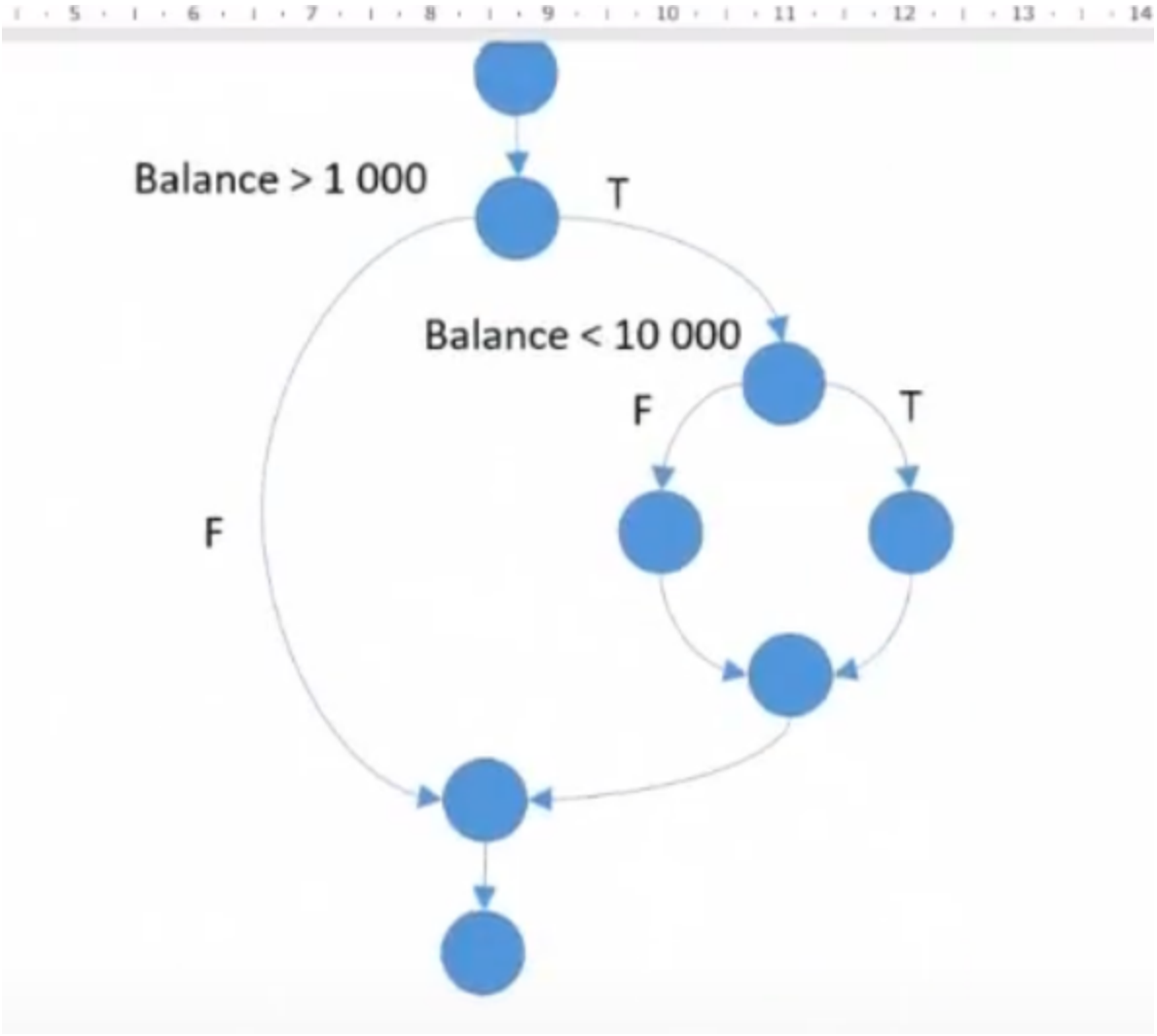
б) Да се опишат основните стъпки при конструиране на графа.

в) Да се дефинират тестови сценарии, като се използва конструираният граф, за да се получи пълно покритие на изразите (statements).

г) Даден е следният псевдо-код:

```
Begin
Read Time
If Time < 12 Then
 Print(Time, "am")
Endif
If Time > 12 Then
 Print(Time -12, "pm")
Endif
If Time = 12 Then
 Print(Time, "noon")
Endif
End
```

Да се определи какво ще бъде тестовото покритие на решенията (decisions) в проценти при тестови сценарии `Time = 11` и `Time = 15`. Отговорът да се обоснове.



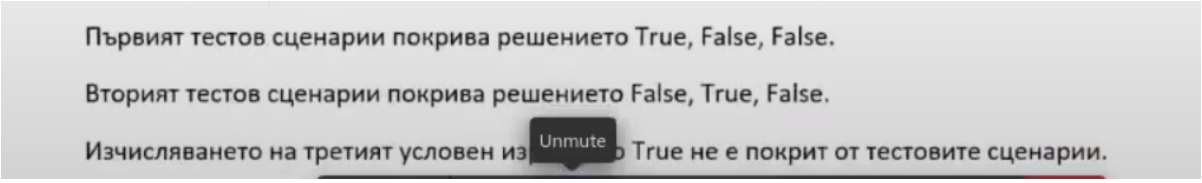
b) Последователност на конструиране на граф на управляващия поток

- Асоцииране на обработващите възли с изразите за **присвояване, извикване на процедури или функции**
- Асоцииране на възлите за взимане на решение с изразите за **условен преход** “if-then-else” или “if-then”, или **множествено разклонение** “switch-case”
- Създаване на специален тип възли за разклонение и асоциирането им с **изразите за цикъл**
- Асоцииране на началния и крайния възел на графа с **първия и последния израз в програмата**

c) От графа на управляващия поток е видно, че лявото разклонение (Balance < 1 000) не се изпълнява. Налични са два алтернативни пътя, при които се изпълняват условията 1 000 < Balance < 10 000 и Balance ≥ 10 000. Следователно за пълно покритие на изразите са необходимо два тестови сценария, например: Balance = 5 000 и Balance = 20 000.

d) Тестовото покритие на решенията (decisions) е 83% (5/6)

Unmute


Първият тестов сценарии покрива решението True, False, False.
Вторият тестов сценарии покрива решението False, True, False.
Unmute
True не е покрит от тестовите сценарии.

Задача 3. Оценка на студент за конкретна дисциплина се изчислява в точки при скала от 1 до 100 като се взимат предвид следните компоненти:

- 20% семинар
- 30% курсов проект
- 50% тест

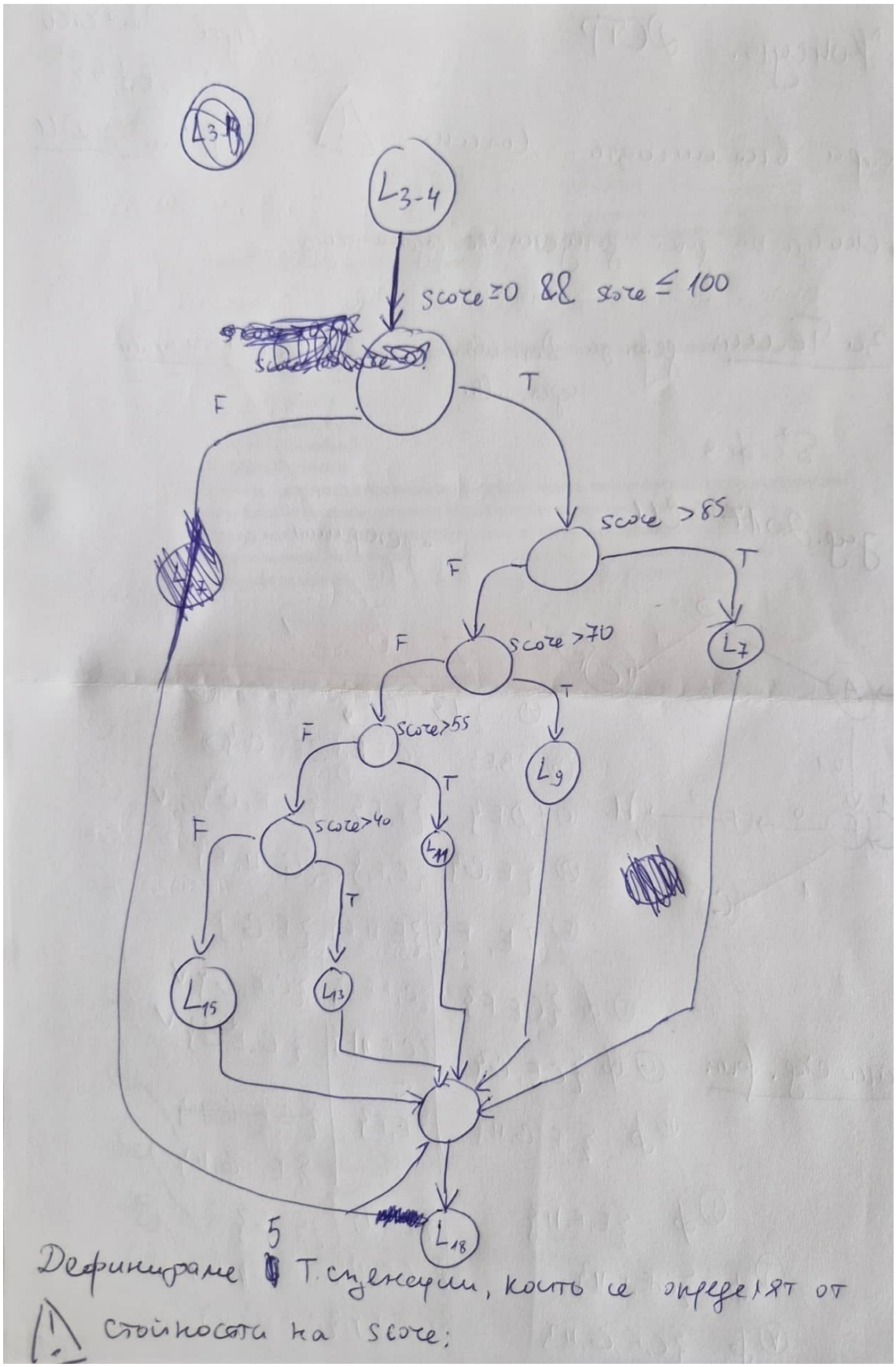
Определянето на крайната оценка въз основа на получените точки е по следната скала:

- ≤ 40 – Слаб 2
- > 40 – Среден 3
- > 55 – Добър 4
- > 70 – Мн. добър 5
- > 85 – Отличен 6

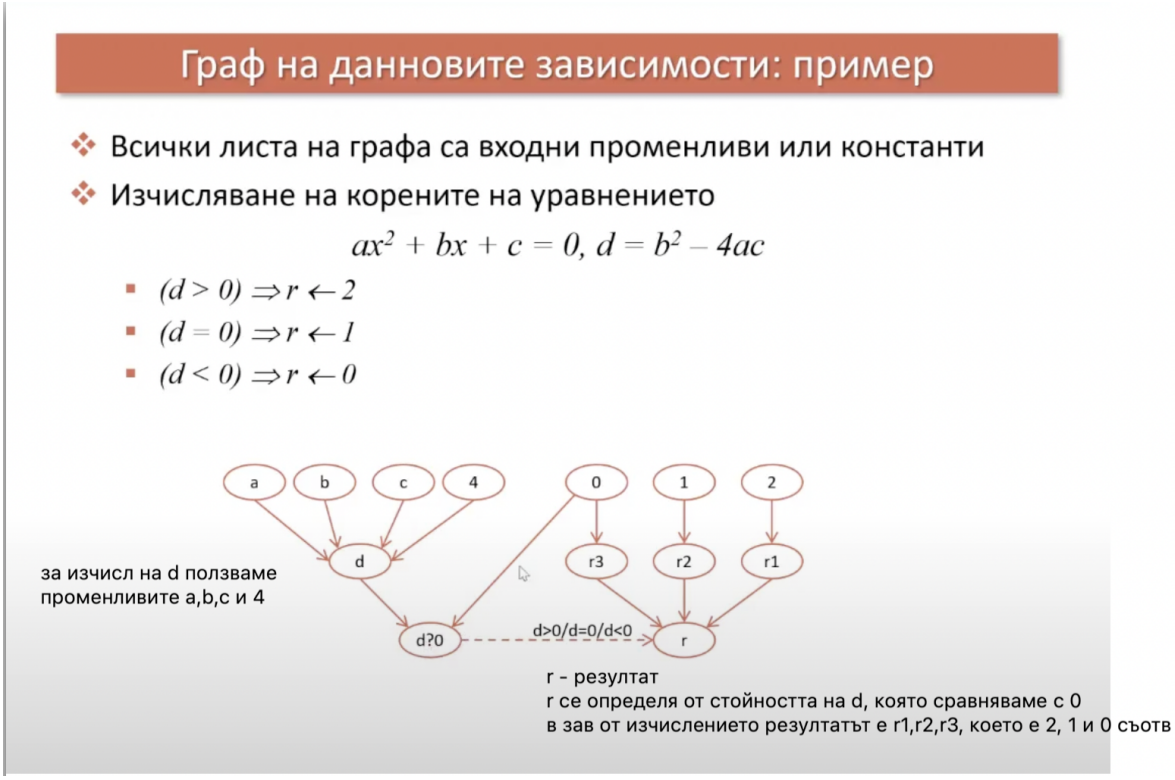
Да се опише с псевдокод изчисляването на крайната оценка, като се обозначат броя на получените точки за всеки от трите компонента на оценката съответно променливите **a**, **b** и **c**. Да се конструира модел с граф на управляващия поток, от който да се дефинират тестови сценарии за структурно тестване (тестване по метода на бялата кутия). Да се опишат основните стъпки при конструиране на модела. ~~Без Фокусване, че точките са цели числа~~

```
line
1   Program determineFinal() {
2   double a, b, c; final;
3       read a, B, c;
4       double score = 0,2* a + 0,3* B + 0,4* c;
5   if(score ≥ 0 && score ≤ 100) {
6       if(score > 85)
7           return 6;          // final ← 6;
8       else if(score > 70)
9           return 5;          // final ← 5;
10      else if(score > 55)
11          return 4;          // final ← 4;
12      else if(score > 40)
13          return 3;          // final ← 3;
14      else
15          return 2;          // final ← 2;
16  }
17  ~~else Input ERROR; return -1;~~ ~~// final ← -1;~~
18  output final;
```

// по-вярно е спокойно да е цяло във final


L3-4
score ≥ 0 && score ≤ 100
T
F
score > 85
F
T
F
score > 70
L7
T
F
score > 55
L9
T
F
score > 40
L11
T
L15
L13
L18
5
Дефинираме 5 Т. функции, които се определят от
стойността на score;

Граф на данновите зависимости - най вероятно няма да го има



За отделните коеф. a,b,c създаваме по един възел.

Присъстват и константите в съответния граф.